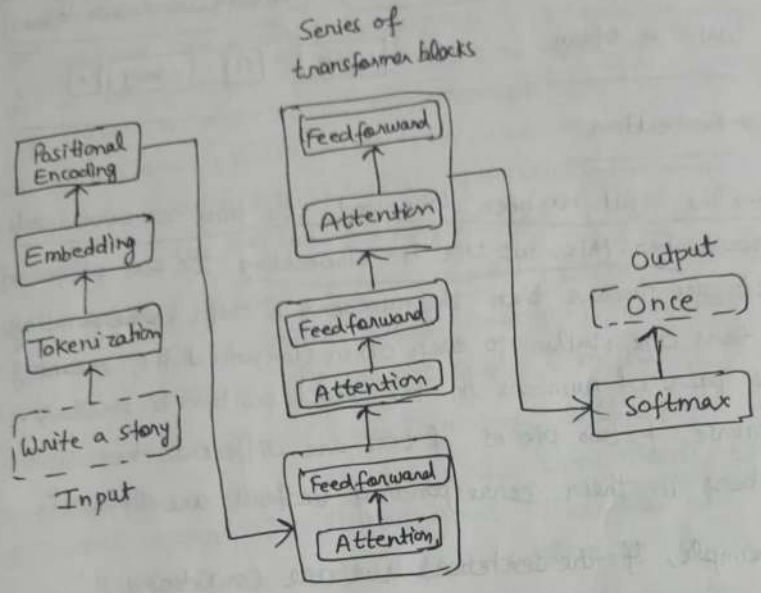


The architecture of transformer model

Tokenization

Tokenization is the most basic step. ~~step~~ It consists of a large dataset of tokens, including all the words, punctuation signs etc. The tokenization step takes every word, prefix, suffix & punctuation signs & sends them to a known token from the library.

Tokenization (Turning words into tokens)

Write a story. ⟶ [Write] [A] [story] [.]

↳ Embedding

Once the input has been tokenized, it's time to turn words into numbers. For this, we use an embedding. If two pieces of text are similar then the numbers in their corresponding vectors are similar to each other (componentwise, meaning each pair of numbers in the same position is similar). Otherwise, if two pieces of text are different then the numbers in their corresponding vectors are different.

For example, if the sentences we are considering is "Write a story", and the tokens are <Write>, <a>, <story>, and <.>, then each one of these will be sent to a long vector & we'll have 4 vectors:

Write ⟶	2.13	-0.42	...	-1.03
A ⟶	0.91	0.56	...	0.23
Story ⟶	-1.56	1.34	...	0.14
. ⟶	1.42	-1.32	...	-2.41

In general embeddings, send every word (token) to a long list of numbers.

⇒ Positional

Once we have t
tokens in the S
into one vector t
bunch of vector
That means, we
For example, if
their correspon
work, but then
that case, the se
the sentence "
same vector, g
in different c
come up wi
vector for the
we'll go wi
Positional enc
predefined v
This ensures
and sentence
be assigned

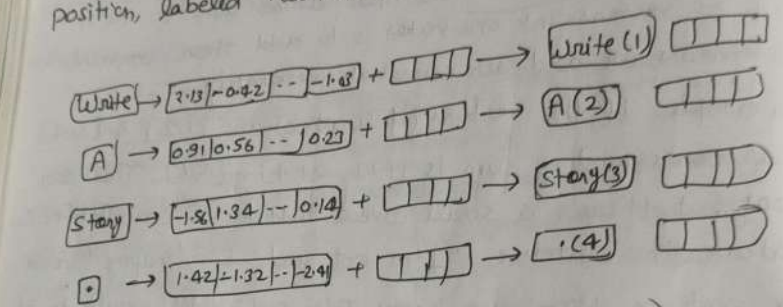
⇒ Positional encoding

Once we have the vectors corresponding to each of the tokens in the sentence, the next step is to turn all these into one vector to process. The most common way to turn a bunch of vectors into one vector is to add them, componentwise. That means, we add each coordinate separately.

For example, if the vectors (of length 2) are [1,2] & [3,4], their corresponding sum is [1+3, 2+4] = [4,6]. This can work, but there's a small caveat. Addition is commutative. In that case, the sentence "I'm not sad, I'm happy" and the sentence "I'm not happy, I'm sad" will result in the same vector, given that they have the same words, except in different order. This is not good. Therefore, we must come up with some method that will give us a different vector for the 2 sentences. Several methods work, and we'll go with one of them: positional encoding.

Positional encoding consists of adding a sequence of predefined vectors to the embedding vectors of the words. This ensures we get a unique vector for every sentence, and sentences with the same words in diff order will be assigned diff vectors.

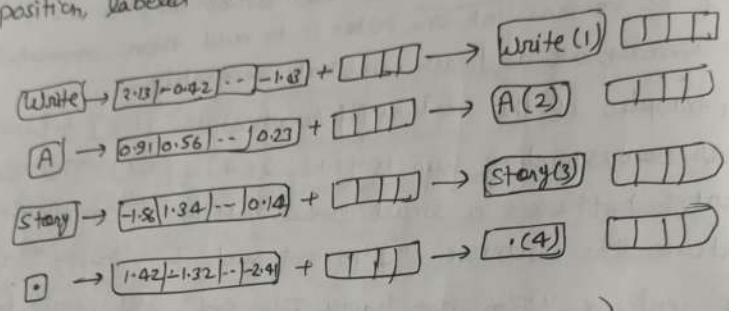
In the example below, the vectors corresponding to the words "Write", "a", "story", and ".", become the modified vectors that carry info^n about their position, labeled "Write(1)", "a(2)", "story(3)", and ".(4)".



Positional encoding adds a position vector to each word, in order to keep track of the positions of the words.

↳ Now that we know, we have a unique vector corresponding to the sentence, & that this vector carries the info^n on all the words in the sentence & their order, we can move to the next step.

In the example below, the vectors corresponding to the words "write", "a", "story", and ".", become the modified vectors that carry info^n about their position, labeled "write(1)", "a(2)", "story(3)", and ".(4)".



Positional encoding adds a position vector to each word, in order to keep track of the positions of the words.

↳ Now that we know, we have a unique vector corresponding to the sentence, & that this vector carries the info^n on all the words in the sentence & their order, we can move to the next step.

⇒ Transformer block

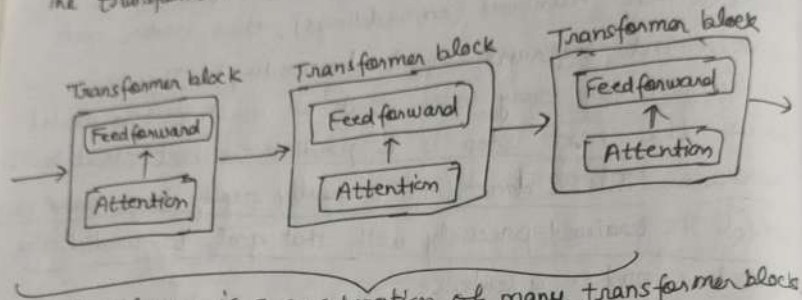
Let's recap what we have so far. The words come in & get turned into tokens (tokenization), tokenized words are turned into numbers (embeddings), then order gets taken into account (positional encoding). This gives us a vector for every token that we input to the model. Now the next step is to predict the next word in this sentence. This is done with a really really large neural n/w, which is trained precisely with that goal, to predict the next word in a sentence.

We can train such a large n/w, but we can vastly improve it by adding a key step: the attention component. Introduced in the seminal paper "Attention is All you Need", it is one of the key ingredients in transformer models, and one of the reasons they work so well. Imagine Attention is a way to add context to each word in the text.

The attention component is added at every block of the feedforward n/w. Therefore, if you imagine a large feedforward neural n/w whose goal is to predict the next word, formed by several blocks of smaller neural n/ws, an attention component is added to each one of these blocks. Each component of the transformer, called a transformer block, is then formed by 2 main components:

- The attention component
- The feedforward "

The transformer is a concatenation of many transformer blocks



The transformer is a concatenation of many transformer blocks. Each ~~of~~ one of these is composed by an attention component followed by a feedforward component (a neural n/w).

⇒ Attention

The next step is attention. It deals with a very important prob^m: The prob^m of context. Sometimes, as you know, the same word can be used with different meanings. This tends to confuse language models, since an embedding simply sends words to vectors, w/o knowing which definition of the word they are using.

Attention is a very powerful & useful technique that helps language models understand the context.

In order to understand how attention works, consider the following 2 sentences:

- Sentence 1: The bank of the river.
- Sentence 2: Money in the bank.

As you can see, the word 'bank' appears in both, but with different definitions. In sentence ①, we are referring to the land at the side of the river & in the 2nd one, to the institution that holds money. The computer has no idea of this, so we need to somehow inject that knowledge into it. What can help us? Well, it seems that the other words in the sentence can come to our rescue. For the 1st sentence, words 'the', and 'of' do us no good. But the word 'river' is the one that is letting us know that we're talking about the land at the side of the river. Similarly, in sentence ②, the word 'money' is the one that is helping us understand that the word 'bank' is now referring to the institution that holds money.

↳ In short, what attention does is, it moves the words in a sentence (or piece of text) closer in the word embedding. In that way, the word "bank" in the sentence "Money in the bank" will be moved closer to the word "money". Equivalently, in the sentence, "The bank of the river", the word "bank" will be moved closer to the word "river".

That way, the modified word "bank" in each of the 2 sentences will carry some of the info^n of the neighboring words, adding context to it.

The attention step used in transformer models is actually much more powerful, and it's called multi-head attention. In multi-head attention, several different embeddings are used to modify the vectors & add context to them. Multi-head attention has helped language models reach much higher levels of efficacy when processing & generating text.

⇒ The Softmax Layer.

Now that you know that a transformer is formed by many layers of transformer blocks, each containing attention & a feedforward layer, you can think of it as a large neural n/w that predicts the next word in a sentence. The transformer outputs scores for all the words, where the highest scores are given to the words that are most likely to be next in the sentence.

The last step of a transformer is a softmax layer, which turns these scores into probabilities (that add to 1), where the highest scores correspond to the highest probabilities. Then, we can sample out of these probs for the next word.